

Obstacles for a Llama

Una llama quiere viajar por la Meseta Andina. Ella tiene un mapa de la meseta en forma de una matriz de $N \times M$ celdas cuadradas. Las filas del mapa están numeradas desde 0 hasta $N - 1$ de arriba hacia abajo y las columnas están numeradas desde 0 hasta $M - 1$ de izquierda a derecha. La celda del mapa en la fila i y la columna j ($0 \leq i < N, 0 \leq j < M$) está denotada por (i, j) .

La llama ha estudiado el clima de la meseta y descubrió que todas las celdas en cada fila del mapa tienen la misma **temperatura** y que todas las celdas en cada columna del mapa tienen la misma **humedad**. La llama tiene dos arreglos de enteros T y H de longitudes N y M , respectivamente. Aquí $T[i]$ ($0 \leq i < N$) indica la temperatura de las celdas en la fila i , y $H[j]$ ($0 \leq j < M$) indica la humedad de las celdas en la columna j .

Además, la llama ha estudiado la flora de la meseta y ha notado que una celda (i, j) está **libre de vegetación** si y solo si su temperatura es mayor que su humedad, formalmente $T[i] > H[j]$.

La llama puede viajar por la meseta siguiendo solo **caminos válidos**. Un camino válido es una secuencia de celdas distintas que satisface las siguientes condiciones:

- Cada par de celdas consecutivas en el camino comparten un lado en común.
- Todas las celdas en el camino están libres de vegetación.

Su tarea es responder Q preguntas. Para cada pregunta, se le dan cuatro enteros: L, R, S y D . Debe determinar si existe un camino válido tal que:

- El camino comienza en la celda $(0, S)$ y finaliza en la celda $(0, D)$.
- Todas las celdas en el camino se encuentran entre las columnas L y R , inclusive.

Se garantiza que tanto la celda $(0, S)$ como la celda $(0, D)$ están libres de vegetación.

Detalles de Implementación

La primera función que debe implementar es:

```
void initialize(std::vector<int> T, std::vector<int> H)
```

- T : un arreglo de longitud N especificando la temperatura en cada fila.
- H : un arreglo de longitud M especificando la humedad en cada columna.

- Esta función es llamada exactamente una vez por cada caso de prueba, antes de cualquier llamada a `can_reach`.

La segunda función que debe implementar es:

```
bool can_reach(int L, int R, int S, int D)
```

- L, R, S, D : enteros que describen una pregunta.
- Esta función es llamada Q veces por cada caso de prueba.

Esta función debe retornar `true` si y solo si existe una camino válido desde la celda $(0, S)$ hasta la celda $(0, D)$, donde todas las celdas en el camino se encuentren entre las columnas L y R , inclusive.

Restricciones

- $1 \leq N, M, Q \leq 200\,000$
- $0 \leq T[i] \leq 10^9$ para cada i tal que $0 \leq i < N$.
- $0 \leq H[j] \leq 10^9$ para cada j tal que $0 \leq j < M$.
- $0 \leq L \leq R < M$
- $L \leq S \leq R$
- $L \leq D \leq R$
- Tanto la celda $(0, S)$ como la celda $(0, D)$ están libres de vegetación.

Subtareas

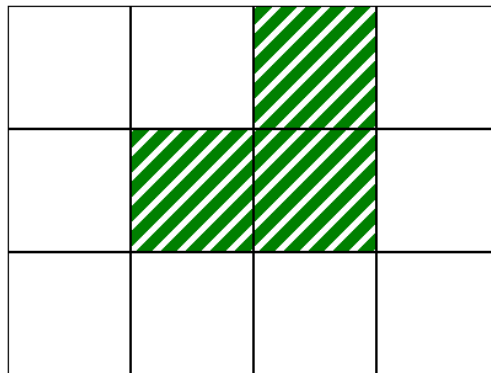
Subtarea	Puntos	Restricciones Adicionales
1	10	$L = 0, R = M - 1$ para cada pregunta. $N = 1$.
2	14	$L = 0, R = M - 1$ para cada pregunta. $T[i - 1] \leq T[i]$ para cada i tal que $1 \leq i < N$.
3	13	$L = 0, R = M - 1$ para cada pregunta. $N = 3$ y $T = [2, 1, 3]$.
4	21	$L = 0, R = M - 1$ para cada pregunta. $Q \leq 10$.
5	25	$L = 0, R = M - 1$ para cada pregunta.
6	17	Sin restricciones adicionales.

Ejemplo

Considere la siguiente llamada:

```
initialize([2, 1, 3], [0, 1, 2, 0])
```

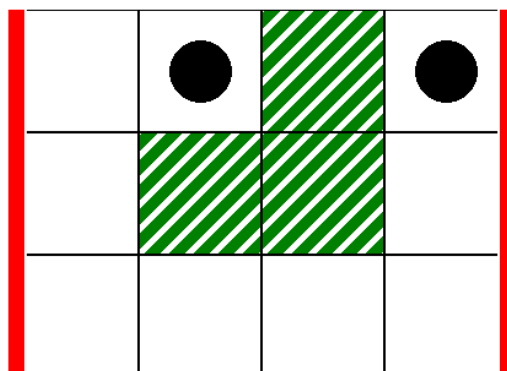
Esto corresponde al mapa en la siguiente imagen, donde las celdas blancas están libres de vegetación.



Para la primera pregunta, se realiza la siguiente llamada:

```
can_reach(0, 3, 1, 3)
```

Esto corresponde al escenario en la siguiente imagen, donde las líneas gruesas indican el rango de las columnas desde $L = 0$ hasta $R = 3$, y los círculos negros indican las celdas de inicio y fin:



En este caso, la llama puede llegar desde la celda $(0,1)$ a la celda $(0,3)$ mediante el siguiente camino válido:

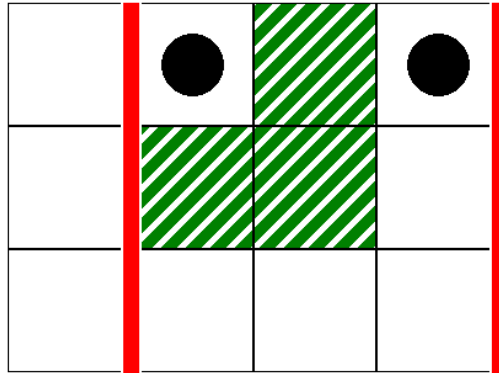
$(0,1), (0,0), (1,0), (2,0), (2,1), (2,2), (2,3), (1,3), (0,3)$

Por tanto, esta llamada debe retornar `true`.

Para la segunda pregunta, se realiza la siguiente llamada:

```
can_reach(1, 3, 1, 3)
```

Esto corresponde al escenario en la siguiente imagen:



En este caso, no existe un camino válido desde la celda $(0, 1)$ hasta la celda $(0, 3)$, tal que todas las celdas en el camino se encuentren entre las columnas 1 y 3, inclusive. Por tanto, esta llamada debe retornar `false`.

Evaluador de Ejemplo

Formato de entrada:

```
N M
T[0] T[1] ... T[N-1]
H[0] H[1] ... H[M-1]
Q
L[0] R[0] S[0] D[0]
L[1] R[1] S[1] D[1]
...
L[Q-1] R[Q-1] S[Q-1] D[Q-1]
```

Aquí, $L[k], R[k], S[k]$ y $D[k]$ ($0 \leq k < Q$) especifican los parámetros para cada llamada a `can_reach`.

Formato de salida:

```
A[0]
A[1]
...
A[Q-1]
```

Aquí, $A[k]$ ($0 \leq k < Q$) es 1 si la llamada `can_reach(L[k], R[k], S[k], D[k])` retorna `true`, y 0 en caso contrario.